

Inspection Process Implementation Guide

FRAPProject / ALBORAK

Working Checklist and Architecture Summary

1. Purpose of this guide

This document summarizes the agreed implementation direction for the new **Inspection Process** module inside the existing ASP.NET Core MVC / EF Core GitHub repository.

It is intended to serve as:

- a reusable implementation checklist
 - an architecture reference
 - a progress checkpoint document
 - a practical guide for future development
-

2. Core strategic decision

We agreed to:

- start the new inspection process **from ground zero**
 - use the old database / Excel process only as **reference**
 - avoid copying the legacy schema blindly
 - build a **clean normalized design**
 - integrate with the **existing repository architecture**
 - start with the **inspection process first**
 - postpone advanced component / serialized / compliance complexity until later phases
-

3. Architectural principles agreed

3.1 Clean separation of concerns

We agreed to separate the solution into these layers:

Entity Models

- Represent database/domain entities
- Should remain as clean POCO classes

- Should not contain repository logic
- Should not contain controller/view logic

EntityConfiguration

- EF Core mapping belongs here
- Keys
- Indexes
- Max lengths
- Relationships
- Delete behavior
- Table mapping

DTO / ViewModels

- Used for MVC controllers and Razor views
- Used for input validation and display
- Can contain:
 - [Required]
 - [Display]
 - [StringLength]
- Must not inherit from repositories or UnitOfWork

Repository / UnitOfWork

- Responsible for data access abstraction
- Works with **entity models only**
- Not with ViewModels/DTOs

3.2 Important rule

DTO/ViewModel classes do not inherit from GenericRepository or UnitOfWork.

Correct:

- repository works with entities
- controller maps entity ↔ viewmodel

Incorrect:

- ViewModel inheriting repository
 - DTO implementing UnitOfWork
-

3.3 LookupBase usage

We agreed to use LookupBase where it truly fits.

Good candidates for LookupBase

- InspectionType
- MaintenanceProgram

Not suitable for LookupBase

Transactional or relational entities such as:

- InspectionTypeProgram
 - ProgramJobCard
 - InspectionState
 - AircraftJobCardState
 - WorkOrder
 - WorkOrderJobCard
 - WorkOrderJobCardSignOff
-

3.4 Delete behavior principle

Because SQL Server rejects many multiple cascade path combinations, we agreed:

Use Restrict / NoAction for:

- master references
- workflow references
- planning state references
- user/audit references

Use Cascade only for true child/owned rows

Examples:

- JobCard -> JobCardAttachment
 - JobCard -> JobCardPlanningRule
 - WorkOrder -> WorkOrderJobCard
 - WorkOrderJobCard -> WorkOrderJobCardSignOff
-

4. Scope chosen for phase 1

We intentionally started with the **inspection-process-first** scope.

Included in phase 1

- inspection definitions
- maintenance programs
- job cards
- planning rules
- aircraft inspection state
- aircraft job-card state
- inspection work orders
- work order job cards
- signoff flow foundation

Deferred to later phases

- deep serialized component management
 - install/remove lifecycle engine
 - advanced compliance / SB logic
 - corrective maintenance workflow
 - advanced extension logic
 - part consumption / parts demand
 - full support/install position model
-

5. Agreed inspection-process domain entities

Master / setup entities

- InspectionType
- MaintenanceProgram
- InspectionTypeProgram
- JobCard
- ProgramJobCard
- JobCardPlanningRule
- JobCardAttachment

Runtime state entities

- InspectionState
- AircraftJobCardState

Workflow entities

- WorkOrder
 - WorkOrderJobCard
 - WorkOrderJobCardSignOff
-

6. Final entity model direction

InspectionType : LookupBase

Includes:

- AcTypeId
- Kind
- interval / tolerance fields
- next inspection self-reference
- audit timestamps

MaintenanceProgram : LookupBase

Includes:

- AcTypeId
- document metadata
- revision metadata
- audit timestamps

JobCard

Standalone entity, not LookupBase Includes:

- AcTypeId
- CardCode
- Title
- technical references
- specialty
- revision info

Planning and workflow entities

Remain standalone POCO entities with navigation properties only.

7. What was completed

Step 4B.1 — Models

Completed:

- initial entity set designed
 - later refactored to align with:
 - LookupBase
 - clean architecture
 - repository/unit-of-work design
-

Step 4B.2 — FRAContext

Completed:

- new `DbSet<>` registrations added for inspection entities
- `ApplyConfiguration(...)` registrations prepared/added

DbSets added

- `InspectionTypes`
 - `MaintenancePrograms`
 - `InspectionTypePrograms`
 - `JobCards`
 - `ProgramJobCards`
 - `JobCardPlanningRules`
 - `JobCardAttachments`
 - `InspectionStates`
 - `AircraftJobCardStates`
 - `WorkOrders`
 - `WorkOrderJobCards`
 - `WorkOrderJobCardSignOffs`
-

Step 4B.3 — EntityConfiguration

Completed and later refactored.

Configurations created

- InspectionTypeConfiguration
- MaintenanceProgramConfiguration
- InspectionTypeProgramConfiguration
- JobCardConfiguration
- ProgramJobCardConfiguration
- JobCardPlanningRuleConfiguration
- JobCardAttachmentConfiguration
- InspectionStateConfiguration
- AircraftJobCardStateConfiguration
- WorkOrderConfiguration
- WorkOrderJobCardConfiguration
- WorkOrderJobCardSignOffConfiguration

Refactor results

- moved mapping concerns fully into Fluent API
 - aligned InspectionType and MaintenanceProgram with LookupBase
 - corrected delete behaviors to avoid SQL Server cascade path failures
-

Step 4B.4 — DTO / ViewModels

Designed for MVC use.

Shared helper ViewModels

- LookupOptionViewModel
- UserLookupViewModel
- AircraftLookupViewModel
- AcTypeLookupViewModel

Feature ViewModels designed

- InspectionType
 - list
 - form

- details
 - MaintenanceProgram
 - list
 - form
 - details
 - JobCard
 - list
 - form
 - details
 - attachment item
 - JobCardPlanningRule
 - list
 - form
 - details
 - WorkOrder
 - list
 - form
 - details
 - job card item
 - signoff item
-

Step 4B.4R — architecture correction

A critical refinement step was completed.

We clarified:

- entities should not be overloaded with validation/mapping concerns
 - DTO/ViewModels belong to UI/application logic
 - repositories belong to infrastructure/data access
 - entity models must remain entity models only
-

Migration and Database Update

Completed successfully.

Achievements

- migration created
- migration reviewed
- SQL Server cascade issues detected
- configurations corrected
- database update completed successfully

Result

The inspection process foundation schema now exists in the database.

8. SQL Server issues encountered and lessons learned

Issue 1

Multiple cascade path involving:

- WorkOrders
- AspNetUsers
- OpenedByUserId
- ClosedByUserId

Issue 2

Multiple cascade path involving:

- AircraftJobCardStates
- Aircrafts
- JobCards
- WorkOrders
- planning rule references

Resolution

Reduce cascading actions:

- avoid multiple `SetNull` / `Cascade` paths from same principal
- favor `Restrict` for user and workflow references

Lesson

In SQL Server:

- `SetNull` is treated as a cascading action

- too many FK cascade paths trigger Error 1785
-

9. Current project state

Already complete

- domain design
- EF Core model layer
- EF Core configuration layer
- DbContext registration
- migration
- database update
- initial DTO/ViewModel design

Not yet implemented

- repositories for the new inspection entities
 - unit of work integration for them
 - controllers
 - Razor views
 - entity ↔ viewmodel mapping logic
 - business services for due calculation / work-order generation
-

10. Recommended next sequence

Next immediate step

Step 4B.5A — Repository + UnitOfWork support for InspectionType

Includes:

- IInspectionTypeRepository
- InspectionTypeRepository
- IUnitOfWork update
- UnitOfWork update

Then

Step 4B.5B — InspectionTypesController

Then

Step 4B.6 — InspectionType Views

- Index
- Create
- Edit
- Details
- Delete

Then repeat in this order

1. MaintenanceProgram
 2. JobCard
 3. JobCardPlanningRule
 4. WorkOrder
-

11. Recommended repository/controller pattern

Correct controller flow

GET Create/Edit

- repository loads supporting entities
- controller maps entity data to ViewModel
- returns ViewModel to Razor view

POST Create/Edit

- controller receives ViewModel
- validates ModelState
- loads entity or creates new entity
- maps ViewModel → entity
- saves via repository + unit of work

Correct repository usage

Repositories should work with:

- `InspectionType`
- `MaintenanceProgram`
- `JobCard`
- `WorkOrder`

Not with:

- `InspectionTypeFormViewModel`
 - `JobCardDetailsViewModel`
 - `WorkOrderFormViewModel`
-

12. Practical implementation checklist

Foundation

- ☒ Define inspection-first domain scope
- ☒ Design entities
- ☒ Refactor entities to align with architecture
- ☒ Use `LookupBase` where appropriate
- ☒ Add `DbSet<>` entries
- ☒ Register entity configurations
- ☒ Create migration
- ☒ Resolve SQL Server cascade-path errors
- ☒ Update database

Application layer

- ☐ Add repository support for `InspectionType`
- ☐ Add `InspectionType` to `UnitOfWork`
- ☐ Create `InspectionTypesController`
- ☐ Add Razor views for `InspectionType`
- ☐ Implement entity ↔ viewmodel mapping
- ☐ Repeat for `MaintenanceProgram`
- ☐ Repeat for `JobCard`
- ☐ Repeat for `JobCardPlanningRule`
- ☐ Repeat for `WorkOrder`

Business logic later

- ☐ Due calculation logic
 - ☐ Planning rule evaluation logic
 - ☐ Work-order generation logic
 - ☐ Signoff validation logic
 - ☐ Inspection closure logic
-

13. Key rules to remember

- Do not let the legacy DB dictate the new design blindly
 - Keep entities clean
 - Keep EF mapping in Fluent API
 - Keep validation in ViewModels
 - Repositories handle entities only
 - Use `LookupBase` only when it genuinely fits
 - Prefer `Restrict` over aggressive cascade behavior in SQL Server workflow models
 - Build incrementally: schema first, then repositories, then controllers, then views
-

14. Final status checkpoint

Current status

Inspection Process Foundation Completed

Next milestone

InspectionType CRUD through Repository + UnitOfWork + Controller + Views

15. Suggested file name

`Inspection-Process-Implementation-Guide.md`

16. Suggested export method

Option A — Markdown to PDF

- open in VS Code / Typora / Obsidian

- print to PDF

Option B — Word

- paste into Microsoft Word
- save as .docx
- export as PDF

Option C — Google Docs

- paste into Google Docs
 - File → Download → PDF
-